

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

GESTIÓN DE ÍNDICES Y VISTAS

En este momento de la clase avanzaremos con los índices y las vistas en un SGBD, y aprenderemos formas para gestionarlos.

Como siempre, es necesario iniciar abordando el contenido teórico para saber sobre lo que hablamos, entonces ¿Qué es un índice? Y una ¿Vista?, en BASES DE DATOS I se realizó una introducción, es momento de profundizar sobre los temas y practicar con algunos ejemplos.

No olvides que este material es creado con el fin de acercarte al contenido, en la bibliografía de la Materia encontrarás en profundidad cada ítem que desarrollamos.

LOS ÍNDICES EN UN SGBD

Los índices son estructuras de datos auxiliares que permiten acelerar el acceso a los datos de una tabla. Funcionan de manera similar a los índices en los libros, al facilitar la localización rápida de registros sin necesidad de realizar un recorrido completo de la tabla.

“Un índice puede ser pensado como una estructura de datos auxiliar que permite un acceso más eficiente a los registros de una tabla, típicamente mediante ordenamiento o hashing.” (Elmasri, R., & Navathe, 2015)

Un índice se crea sobre una o más columnas, y puede ser de distintos tipos como los **B-tree** (los más comunes), **únicos** (garantizan que no haya valores duplicados), **compuestos** (sobre múltiples columnas) y **hash**.

La gestión de índices implica su creación, modificación y eliminación, lo cual debe hacerse considerando el equilibrio entre velocidad de lectura y costo en las operaciones de escritura (inserción, actualización y eliminación), ya que los índices deben mantenerse actualizados.

La creación y eliminación de índices se realiza mediante instrucciones específicas en SQL. Es importante considerar que, aunque los índices mejoran el rendimiento de lectura, también implican un costo adicional en las operaciones de escritura (INSERT, UPDATE y DELETE), ya que deben mantenerse actualizados.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

CREAR Y ELIMINAR ÍNDICES EN SQL

Para crear o eliminar índices en SQL utilizamos los comandos **CREATE INDEX** y **DROP INDEX**, cada uno tiene su función específica:

// Crear un índice //

```
CREATE INDEX idx_apellido ON personas(apellido);
```

// Eliminar un índice //

```
DROP INDEX idx_apellido;
```

En el primer ejemplo, el comando **CREATE INDEX** le dice al sistema de base de datos creá un índice llamado **idx_apellido** sobre la columna **apellido** de la **tabla personas**, lo que nos sirve para que las búsquedas por **apellido** (por ejemplo, cuando hacemos un **SELECT** con **WHERE** apellido = 'Gómez') sean más rápidas, entonces en lugar de revisar fila por fila, el sistema usa el índice como una especie de "lista ordenada" que agiliza la búsqueda.

En el segundo caso, el comando **DROP INDEX** le dice al SGBD eliminá el índice que se llama **idx_apellido**, lo cual es muy útil si nos damos cuenta que ya no necesitamos el índice, o si estamos haciendo muchos cambios en los datos y el índice está ralentizando esas operaciones.

UN CASO TÍPICO DE ÍNDICES: LA BUSQUEDA DE LIBROS

Buscar y encontrar físicamente un libro en una biblioteca enorme es una tarea compleja, por ejemplo, si queremos hallar todos los libros escritos por *Elmasri*¹, y no hay ningún orden especial, ni ayuda, tenemos que mirar libro por libro, uno por uno en todos los estantes hasta encontrarlos, hacerlo nos llevaría mucho tiempo, un índice nos vendría de gran ayuda para agilizar la tarea.

En su forma analógica en la entrada de la biblioteca podríamos tener una lista alfabética de autores, y debajo de cada nombre aparece consignada la estantería donde se ubican sus libros, eso es un índice y nos ayudaría a encontrar rápidamente lo que buscamos sin tener que recorrer todos los estantes.

Pero podría ocurrir que alguien decida quitar esa lista de autores eliminando el índice, el resultado como al principio será que la próxima vez que busquemos un libro por su autor, vamos a tener sin remedio que revisar estante por estante.

¹ **Ramez Aziz Elmasri**. Científico referente de los sistemas de bases de datos, fue profesor en la Universidad de Texas en Arlington. Falleció el 14 de mayo de 2022.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

TIPOS DE ÍNDICES: B-TREE, ÚNICOS, COMPUESTOS, HASH.

B-TREE

B-TREE es el tipo de índice más utilizado por los SGBD relacionales como MySQL, PostgreSQL y Oracle. Organiza los datos en forma de árbol balanceado, lo que permite búsquedas, inserciones y eliminaciones eficientes.

El índice **B-TREE**, es ideal para columnas que se usan en condiciones de búsqueda (**WHERE**), ordenamiento (**ORDER BY**) o rangos (**BETWEEN**, **>**, **<**).

EJEMPLO

Buscar todos los empleados con salario mayor a \$500.000,00.

ÚNICO

El índice **ÚNICO** no permite valores duplicados en la columna o combinación de columnas sobre la que se aplica, se usa para garantizar unicidad en datos importantes, como el número de DNI, correo electrónico o nombre de usuario.

EJEMPLO

Asegurar que no haya dos clientes con el mismo número de documento.

COMPUESTO

El índice **COMPUESTO** (o **Multicolumna**), se crea sobre más de una columna, lo cual mejora el rendimiento de consultas que filtran por varias columnas al mismo tiempo.

Es importante destacar que el orden de las columnas en el índice es significativo para su eficiencia.

EJEMPLO

Buscar pedidos por *cliente_id* y *fecha*.

// Crear un índice para realizar la búsqueda//

CREATE INDEX idx_cliente_fecha **ON** pedidos(cliente_id, fecha);

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

HASH

El índice **HASH**, utiliza una función hash para asignar una clave a los valores indexados, ofreciendo búsquedas muy rápidas para valores exactos (=), pero no sirve para rangos ni ordenamientos.

Los índices **HASH** en PostgreSQL no se usan por defecto, hay que indicarlo explícitamente con **USING HASH**. Algunos SGBD no permiten crear índices **HASH** de forma directa o los implementan internamente sin que el usuario lo defina.

EJEMPLO

Crear un índice **HASH** llamado **idx_codigo_hash** sobre la columna **codigo** para búsquedas exactas de productos por su código.

Primero crearemos la tabla **productos**

```
CREATE TABLE productos (  
    id SERIAL PRIMARY KEY,  
    codigo VARCHAR(20) NOT NULL,  
    nombre VARCHAR(100),  
    precio NUMERIC(10, 2)  
);
```

Luego en la tabla **productos**, el índice para realizar la búsqueda.

```
CREATE INDEX idx_codigo_hash ON productos USING HASH (codigo);
```

Finalmente buscaremos un producto por su código exacto.

```
SELECT * FROM productos  
  
WHERE codigo = 'P12345';
```

El resultado será que veremos todos los productos cuyo código sea igual a 'P12345'.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

UN DETALLE PARA OBSERVADORES...

En este punto, seguramente te estarás preguntando ¿Qué ventajas o diferencias existen entre usar únicamente el comando **SELECT * FROM** con el comando **HASH** y sin él en nuestro SGBD?

Si ejecutamos **con** y **sin** el índice **HASH** creado previamente la consulta

```
SELECT * FROM productos WHERE codigo = 'P12345';
```

¿QUÉ HACE EL MOTOR DE BASE DE DATOS EN CADA CASO?

☒ Con índice HASH creado (USING HASH)

Primeramente, crearemos el índice HASH en la tabla productos

```
CREATE INDEX idx_codigo_hash ON productos USING HASH (codigo);
```

Luego ejecutamos nuestra consulta de búsqueda

```
SELECT * FROM productos WHERE codigo = 'P12345';
```

Como resultado el motor de PostgreSQL

Accede directamente al valor **HASH** de 'P12345' en la estructura del índice, encontrando rápidamente el registro correspondiente sin escanear toda la tabla, siendo muy eficiente en consultas de igualdad exacta, como =.

VENTAJA: velocidad mejorada para búsquedas exactas.

☐ Sin ningún índice HASH creado

Como resultado el motor de PostgreSQL

Hace un escaneo completo (*secuencial scan*) de la tabla y luego revisa fila por fila hasta encontrar coincidencias.

Este método es mucho más lento, sobre todo si la tabla tiene muchos registros.

DESVENTAJA: baja eficiencia en tablas grandes.

¿INTEGRAMOS CONTENIDOS?

Si queremos darnos cuenta el impacto concreto que producen en nuestro SGBD los comandos de esta clase, recordemos lo **estudiado anteriormente** cuando trabajamos en el análisis de la eficiencia con los **planes de ejecución**.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

LAS VISTAS EN UN SGBD

Las vistas son **objetos virtuales** que resultan de una consulta almacenada en el SGBD. **Se comportan como si fueran tablas, pero no almacenan datos de forma permanente.** Su objetivo principal es facilitar la reutilización de consultas complejas, restringir el acceso a ciertos datos y proporcionar diferentes perspectivas de la misma base de datos.

Existen **vistas simples** (basadas en una sola tabla sin funciones agregadas ni agrupamientos), **vistas complejas** (basadas en múltiples tablas, uniones, funciones agregadas, etc.) y **vistas materializadas**, que sí almacenan los resultados y requieren mecanismos de actualización periódica.

La **vista simple** es como un formulario de consultas de información, donde no vemos toda la base de datos, sino solo las columnas o filas de alguna tabla en particular, ocultando los detalles innecesarios mostrando los datos sin poder modificarlos.

La **vista compleja** en tanto combina varias tablas o usa cálculos como sumas o promedios que sirve para mostrar información que ya fue relacionada y procesada, pero no se puede modificar directamente.

Una **vista materializada** es como una fotografía de los datos que se guarda para consultar más rápido, pero no se actualiza automáticamente como sí sucede en una **vista normal** que siempre muestra lo que hay en tiempo real, pero puede tardar más en mostrarse

La gestión de vistas se realiza mediante instrucciones de **creación, modificación y eliminación**. También es necesario controlar los permisos de acceso y considerar la actualización de vistas si se modifican las tablas subyacentes.

CREACIÓN VISTAS EN UN SGBD

Para crear una vista utilizamos el comando **CREATE VIEW**, lo que nos dará como resultado una tabla que no almacena datos, simplemente los muestra, pero como vimos en el apartado anterior, esto tiene sus diferencias según sean **vistas complejas** o **vistas materializadas**.

Una vista se puede usar como si fuera una tabla: `SELECT * FROM vista_empleados_publica;`

Algunas vistas no se pueden actualizar directamente, especialmente si incluyen agregados, GROUP BY, o JOIN complejos. salvo que se defina como materializada.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

CREANDO VISTAS EN SQL

Tenemos múltiples opciones para visualizar datos en nuestras tablas a través de las vistas, las más comunes son las *simples con una sola tabla*, con *alias y filtros*, con *funciones agregadas*, las que tienen *JOIN* (varias tablas) y las que nos *permiten proteger datos sensibles*.

EJEMPLOS DE VISTAS EN SQL

VISTAS SIMPLES EN UNA SOLA TABLA

Aquí veremos un ejemplo que nos muestre solo algunos campos de la tabla clientes.

```
CREATE VIEW vista_clientes AS
```

```
SELECT id, nombre, ciudad
```

```
FROM clientes;
```

Si la tabla **clientes** tiene los siguientes datos

id	nombre	ciudad	email	telefono
1	Laura González	Córdoba	laura@mail.com	3511234567
2	Martín Pérez	Rosario	martinp@mail.com	3419876543
3	Ana Torres	Buenos Aires	ana.torres@mail.com	1145567788

Y ejecutamos la consulta SQL, obtendremos la siguiente **visualización simple**

vista_clientes		
id	nombre	ciudad
1	Laura González	Córdoba
2	Martín Pérez	Rosario
3	Ana Torres	Buenos Aires

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

VISTAS CON ALIAS Y FILTROS

En este caso haremos una vista de productos activos con alias para los nombres de columna, donde la vista solo incluye los productos que están activos y cambia los nombres de algunas columnas usando alias (AS) para hacerlos más breves o claros.

```
CREATE VIEW vista_productos_activos AS
```

```
SELECT codigo AS cod, nombre AS producto, precio
```

```
FROM productos
```

```
WHERE activo = TRUE;
```

Si la tabla **productos** tiene los siguientes datos

codigo	nombre	precio	activo
UTN001	Mouse óptico	3500	TRUE
UTN002	Teclado mecánico	9500	FALSE
UTN003	Monitor 24"	48000	TRUE
UTN004	Cable HDMI	1200	TRUE

Y ejecutamos la consulta SQL, obtendremos la siguiente **visualización con alias y filtros**

vista_productos_activos		
cod	producto	precio
UTN001	Mouse óptico	3500
UTN003	Monitor 24"	48000
UTN004	Cable HDMI	1200

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

VISTAS CON FUNCIONES AGREGADAS

Ahora veremos como mostrar el total de ventas y el monto acumulado por cliente.

```
CREATE VIEW vista_ventas_por_cliente AS
```

```
SELECT cliente_id, COUNT(*) AS total_ventas, SUM(monto) AS total_monto
```

```
FROM ventas
```

```
GROUP BY cliente_id;
```

Si la tabla **ventas** tiene los siguientes datos

id_venta	cliente_id	monto	fecha
1	101	15000	15/1/2024
2	102	8900	17/1/2024
3	101	23000	20/1/2024
4	103	4500	22/1/2024
5	102	12000	23/1/2024
6	101	10000	30/1/2024

Y ejecutamos la consulta SQL, obtendremos la siguiente **visualización con funciones**

vista_ventas_por_cliente		
cliente_id	total_ventas	total_monto
101	3	48000
102	2	20900
103	1	4500

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

VISTAS QUE PROTEGEN DATOS SENSIBLES

Podemos también querer ocultar columnas sensibles de nuestro origen de datos como sueldo o documento.

```
CREATE VIEW vista_empleados_publica AS
```

```
SELECT nombre, puesto, departamento
```

```
FROM empleados;
```

Si la tabla **empleados** tiene los siguientes datos

Id_em	nombre	puesto	area	salario	email
1	Carla Ramírez	Analista	IT	250000	carla@utn.com
2	Julián Díaz	Programador	IT	300000	julian@utn.com
3	Marta López	RRHH	R.R.H.H.	200000	marta@utn.com
4	Pedro Martínez	Contador	Finanzas	280000	pedro@utn.com

Y ejecutamos la consulta SQL, obtendremos la siguiente **visualización con protección**

vista_empleados_publica		
nombre	puesto	area
Carla Ramírez	Analista	IT
Julián Díaz	Desarrollador	IT
Marta López	RRHH	R.R.H.H.
Pedro Martínez	Contador	Finanzas

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

VISTAS UTILIZANDO UNIONES O JOINS

Si poseemos datos de múltiples tablas, podemos unir varias para mostrar el detalle de cada pedido.

```
CREATE VIEW vista_detalle_pedidos AS
```

```
SELECT p.id_pedido, c.nombre AS cliente, pr.nombre AS producto, dp.cantidad
```

```
FROM pedidos p
```

```
JOIN clientes c ON p.cliente_id = c.id
```

```
JOIN detalle_pedidos dp ON p.id_pedido = dp.pedido_id
```

```
JOIN productos pr ON dp.producto_id = pr.id;
```

Si las tablas **clientes**, **productos**, **pedidos** y **detalle_pedidos** tienen los siguientes datos

clientes	
id	nombre
1	Ana Torres
2	Luis Gómez

productos	
id	nombre
10	Notebook
11	Impresora

pedidos		
id_pedido	cliente_id	fecha
1001	1	1/3/2024
1002	2	5/3/2024

detalle_pedidos		
pedido_id	producto_id	cantidad
1001	10	2
1001	11	1
1002	11	3

Y ejecutamos la consulta SQL, obtendremos la siguiente **visualización con unión**

Vista_detalle_pedidos			
id_pedido	cliente	producto	cantidad
1001	Ana Torres	Notebook	2
1001	Ana Torres	Impresora	1
1002	Luis Gómez	Impresora	3

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

PASANDO EN LIMPIO

CARACTERÍSTICA	VISTA SIMPLE	VISTA COMPLEJA	VISTA MATERIALIZADA
Basada en una sola tabla	SI	NO (involucra varias tablas o subconsultas)	SI/ NO (puede ser compleja o simple)
Incluye funciones agregadas	NO	SI	SI
Permite modificar datos	SI (si no hay restricciones)	NO (usualmente no)	NO
Almacena datos físicamente	NO (se calcula en cada consulta)	NO (igual que la simple)	SI (almacena datos en disco)
Mejora el rendimiento	NO	NO	SI (evita recalcular resultados complejos)
Actualización de datos	Siempre refleja los datos actuales	Igual que la simple	Requiere actualización manual o programada

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

TRABAJAMOS

Es momento de hacer algunas prácticas en SQL con la operación de creación de vistas.

Presta atención a la consigna y realizá la práctica.

¡Manos a la obra!

ACTIVIDAD DE PRÁCTICA N° 1: VISTA SIMPLE

En esta primera práctica de la tabla empleados vas a crear una vista que seleccione columnas específicas.

Iniciamos con la creación de la tabla **empleados**

```
CREATE TABLE empleados (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50),  
    puesto VARCHAR(50),  
    salario DECIMAL(10,2),  
    departamento VARCHAR(50)  
);
```

Luego insertamos datos de ejemplo

```
INSERT INTO empleados VALUES  
(1, 'Ana Torres', 'Analista', 75000, 'Finanzas'),  
(2, 'Carlos Pérez', 'Desarrollador', 68000, 'TI'),  
(3, 'Lucía Gómez', 'Soporte', 50000, 'TI');
```

¿QUE SE TE PIDE?

Crear una vista llamada **vista_empleados_ti** que muestre **nombre** y **puesto** solo de **empleados** del **departamento TI**.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

ACTIVIDAD DE PRÁCTICA N° 2: VISTA COMPLEJA

En esta segunda práctica vas a crear una vista basada en las tablas clientes y ventas utilizando funciones de agregación.

Presta atención a la consigna y realizá la práctica.

¡Manos a la obra!

Iniciamos con la creación de las tablas **clientes** y **ventas**

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50)  
);
```

```
CREATE TABLE ventas (  
    id INT PRIMARY KEY,  
    cliente_id INT,  
    monto DECIMAL(10,2),  
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)  
);
```

Luego insertamos datos de ejemplo

```
INSERT INTO clientes VALUES (1, 'Laura Díaz'), (2, 'Tomás Ruiz');  
  
INSERT INTO ventas VALUES  
(1, 1, 120.00), (2, 1, 300.50), (3, 2, 500.00);
```

¿QUE SE TE PIDE?

Crear una vista llamada **vista_ventas_por_cliente** que muestre por cada cliente: su **ID**, la **cantidad de ventas realizadas** y el **monto total**

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

ACTIVIDAD DE PRÁCTICA N° 3: VISTA MATERIALIZADA

En esta tercera práctica vas a crear una vista materializada que almacene los datos y no dependa del estado actual de las tablas.

¡ATENCIÓN!

Es importante saber que para poder realizar la práctica en SQLWorkbench es necesario tener instalado el motor PostgreSQL ya que MySQL estándar no te permitirá ejecutar la consulta.

Presta atención a la consigna y realizá la práctica.

¡Manos a la obra!

¿QUE SE TE PIDE?

Crear una vista materializada de las tablas **clientes y ventas**, luego crea la consulta que te permita ver los datos (*capturá la pantalla con el resultado para que te sirva de guía*).

Luego registra una nueva venta y comprobar que no se refleja automáticamente el resultado en la vista (*captura la pantalla con el resultado para que te sirva de guía*).

Compará las dos capturas de pantalla, y explica lo que ves en cada una con la consulta que pide la visualización realizada.

BASE DE DATOS II**CONTENIDOS Y ACTIVIDADES****COMPARACIONES ENTRE CONSULTAS CON Y SIN ÍNDICES EN UN SGBD**

Comenzaremos recordando que un índice en una base de datos es una estructura auxiliar que mejora el rendimiento de las consultas, permitiendo buscar registros de manera más rápida, similar al índice de un libro que nos guía directamente hacia una sección específica sin leer todas las páginas.

¿Qué sucede cuando no usamos índices? Cuando se realiza una consulta sin que exista un índice sobre la columna utilizada en el filtro, por ejemplo, en una cláusula WHERE, el motor de la base de datos debe examinar fila por fila, lo que se denomina un escaneo completo de tabla o full table scan. Esto puede implicar un costo computacional elevado, sobre todo en tablas grandes.

¿Qué cambia cuando usamos un índice? Si la columna utilizada en la condición tiene un índice creado, el SGBD utiliza dicho índice para localizar rápidamente el valor buscado. Esto se traduce en tiempos de respuesta más rápidos, especialmente en tablas con gran volumen de datos.

PASANDO EN LIMPIO

CRITERIO	CONSULTA SIN ÍNDICE	CONSULTA CON ÍNDICE
Tiempo de búsqueda	Lento (escaneo completo de tabla)	Rápido (acceso directo)
Uso de recursos	Alto (CPU y disco)	Menor
Utilización en tablas chicas	Diferencia poco notable	Puede no justificar su uso
Utilización en tablas grandes	Muy ineficiente	Alta mejora en rendimiento
Mantenimiento del índice	No requiere	Requiere espacio y actualización

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

PRÁCTICA DE CONSULTAS CON Y SIN INDICES

ACTIVIDAD PRÁCTICA N° 4

SIN ÍNDICE

Primeramente, creamos la base de datos **bd_ejemplo_indices**

```
CREATE DATABASE bd_ejemplo_indices;
```

Luego usaremos la base de datos **bd_ejemplo_indices**

```
USE bd_ejemplo_indices;
```

Seguidamente la tabla **productos**

```
CREATE TABLE productos (  
    codigo VARCHAR(10) PRIMARY KEY,  
    nombre VARCHAR(100),  
    categoria VARCHAR(50),  
    precio DECIMAL(10,2),  
    stock INT  
);
```

Luego le añadimos algunos registros iniciales para poder trabajar,

¡ATENCIÓN!

Cómo ayuda en el material complementario de esta clase encontrarás dos archivos **.SQL** listos para trabajar.

```
INSERT INTO productos (codigo, nombre, categoria, precio, stock) VALUES
```

```
('P001', 'Notebook Lenovo', 'Tecnología', 450000.00, 12),
```

```
('P002', 'Smartphone Samsung', 'Tecnología', 300000.00, 25),
```

```
('P003', 'Teclado inalámbrico', 'Accesorios', 15000.00, 50),
```

```
('P004', 'Mouse óptico', 'Accesorios', 10000.00, 60),
```

```
('P005', 'Monitor 24 pulgadas', 'Tecnología', 120000.00, 8),
```

```
('P006', 'Impresora HP', 'Oficina', 80000.00, 10),
```

```
('P007', 'Silla ergonómica', 'Muebles', 90000.00, 5),
```

```
('P008', 'Router WiFi', 'Tecnología', 25000.00, 20),
```

```
('P009', 'Disco Externo 1TB', 'Accesorios', 35000.00, 18),
```

```
('P010', 'Webcam HD', 'Accesorios', 20000.00, 30);
```

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

Ahora probaremos nuestra B.D. con una consulta específica a la tabla **productos**

```
SELECT * FROM productos WHERE codigo = 'P001';
```

Podemos probar el impacto del uso de índices

```
CREATE INDEX idx_codigo ON productos(codigo);
```

¿CUAL ES LA CONCLUSIÓN?

Si no hay un índice sobre la columna **codigo**, el SGBD recorre todos los registros de la tabla **productos** para encontrar coincidencias.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

ACTIVIDAD PRÁCTICA N° 5

CON ÍNDICE

Teniendo de base la colección de datos **bd_ejemplo_indices** creada anteriormente

Creamos el índice **idx_codigo** en la tabla **productos**, aplicando el índice sobre la **columna codigo**.

```
CREATE INDEX idx_codigo ON productos(codigo);
```

Luego de la tabla **productos** buscamos todos los que sean **igual** a **P001**

```
SELECT * FROM productos WHERE codigo = 'P001';
```

¿CUAL ES LA CONCLUSIÓN?

En este caso, el índice **idx_codigo** permite al SGBD acceder directamente a las filas que coinciden con el valor **'P001'**, sin recorrer toda la tabla.

Al buscar un producto por su código (por ejemplo, WHERE codigo = 'P001'), el motor de base de datos no recorre todos los registros, sino que usa el índice para ir directamente al registro que corresponde, como si consultara un índice alfabético en un libro.

Recordemos que el índice es una estructura adicional, como una tabla pequeña o auxiliar ordenada, que permite acceder más rápido a los datos.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

SOLUCIÓN DE LOS TRABAJOS PRÁCTICOS

ACTIVIDAD DE PRÁCTICA N° 1: VISTA SIMPLE

Crear una vista llamada **vista_empleados_ti** que muestre **nombre** y **puesto** solo de **empleados** del **departamento TI**.

Paso 1: Creamos la vista

```
CREATE VIEW vista_empleados_ti AS
```

```
SELECT nombre, puesto
```

```
FROM empleados
```

```
WHERE departamento = 'TI';
```

Paso 2: Ejecutamos la consulta para ver los datos

```
SELECT * FROM vista_empleados_ti;
```

Resultado

vista_empleados_ti	
nombre	puesto
Carlos Pérez	Desarrollador
Lucía Gómez	Soporte

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

SOLUCIÓN DE LOS TRABAJOS PRÁCTICOS

ACTIVIDAD DE PRÁCTICA N° 2: VISTA COMPLEJA

Crear una vista llamada **vista_ventas_por_cliente** que muestre por cada cliente: su **ID**, la **cantidad de ventas realizadas** y el **monto total**.

Paso 1: Creamos la vista

```
CREATE VIEW vista_ventas_por_cliente AS  
  
SELECT cliente_id, COUNT(*) AS total_ventas, SUM(monto) AS total_monto  
  
FROM ventas  
  
GROUP BY cliente_id;;
```

Paso 2: Ejecutamos la consulta para ver los datos

```
SELECT * FROM vista_ventas_por_cliente;
```

Resultado

vista_ventas_por_clientes		
cliente_id	total_ventas	total_monto
1	2	420.50
2	1	500.00

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

SOLUCIÓN DE LOS TRABAJOS PRÁCTICOS

ACTIVIDAD DE PRÁCTICA N° 3: VISTA MATERIALIZADA

Si estamos utilizando SQLWorkBench primero comprobar que el motor SQL sea Postgree.

Crear una vista materializada de las tablas **clientes** y **ventas**, luego crea la consulta que te permita ver los datos (*capturá la pantalla con el resultado para que te sirva de guía*).

Luego registra una nueva venta y comprobar que no se refleja automáticamente el resultado en la vista (*captura la pantalla con el resultado para que te sirva de guía*).

Compará las dos capturas de pantalla, y explica lo que ves en cada una con la consulta que pide la visualización realizada.

Paso 1: Creamos la vista

```
CREATE VIEW vista_ventas_por_cliente AS
SELECT cliente_id, COUNT(*) AS total_ventas, SUM(monto) AS total_monto
FROM ventas
GROUP BY cliente_id;;
```

Paso 2: Ejecutamos la consulta para ver los datos

```
SELECT * FROM vista_ventas_materializada;
```

Resultado antes de efectuar el refresco (REFRESH)

vista_ventas_materializada		
cliente_id	total_ventas	total_monto
1	2	420.50
2	1	500.00

Paso 3: Agregamos una nueva venta

```
INSERT INTO ventas VALUES (4, 1, 150.00);
```

Paso 4: Ejecutamos la consulta nuevamente sin refrescar (REFRESH) la vista

```
SELECT * FROM vista_ventas_materializada;
```

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

Resultado antes de efectuar el refresco (REFRESH), la visualización no cambia.

vista_ventas_materializada		
cliente_id	total_ventas	total_monto
1	2	420.50
2	1	500.00

Paso 5: ejecutamos el refresco (REFRESH)

REFRESH MATERIALIZED VIEW vista_ventas_materializada;

El resultado luego de efectuar el refresco (REFRESH) se muestra actualizado.

vista_ventas_materializadac		
cliente_id	total_ventas	total_monto
1	3	570.50
2	1	500.00

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

SOLUCIÓN DE LOS TRABAJOS PRÁCTICOS

ACTIVIDAD DE PRÁCTICA N° 4: SIN ÍNDICE

Primeramente, creamos la base de datos **bd_ejemplo_indices**

```
CREATE DATABASE bd_ejemplo_indices;
```

Luego usaremos la base de datos **bd_ejemplo_indices**

```
USE bd_ejemplo_indices;
```

Seguidamente la tabla **productos**

```
CREATE TABLE productos (  
    codigo VARCHAR(10) PRIMARY KEY,  
    nombre VARCHAR(100),  
    categoria VARCHAR(50),  
    precio DECIMAL(10,2),  
    stock INT  
);
```

Luego le añadimos algunos registros iniciales para poder trabajar.

¡ATENCIÓN!

Cómo ayuda en el material complementario de esta clase encontrarás dos archivos **.SQL** listos para trabajar.

INSERT INTO productos (codigo, nombre, categoria, precio, stock) VALUES

```
('P001', 'Notebook Lenovo', 'Tecnología', 450000.00, 12),  
( 'P002', 'Smartphone Samsung', 'Tecnología', 300000.00, 25),  
( 'P003', 'Teclado inalámbrico', 'Accesorios', 15000.00, 50),  
( 'P004', 'Mouse óptico', 'Accesorios', 10000.00, 60),  
( 'P005', 'Monitor 24 pulgadas', 'Tecnología', 120000.00, 8),  
( 'P006', 'Impresora HP', 'Oficina', 80000.00, 10),  
( 'P007', 'Silla ergonómica', 'Muebles', 90000.00, 5),  
( 'P008', 'Router WiFi', 'Tecnología', 25000.00, 20),  
( 'P009', 'Disco Externo 1TB', 'Accesorios', 35000.00, 18),  
( 'P010', 'Webcam HD', 'Accesorios', 20000.00, 30);
```


BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

El resultado

productos				
Código	nombre	categoria	precio	stock
P001	Notebook Lenovo	Tecnología	450000.00	12
P002	Smartphone Samsung	Tecnología	300000.00	25
P003	Teclado inalámbrico	Accesorios	15000.00	50
P004	Mouse óptico	Accesorios	10000.00	60
P005	Monitor 24 pulgadas	Tecnología	120000.00	8
P006	Impresora HP	Oficina	80000.00	10
P007	Silla ergonómica	Muebles	90000.00	5
P008	Router WiFi	Tecnología	25000.00	20
P009	Disco Externo 1TB	Accesorios	35000.00	18
P010	Webcam HD	Accesorios	20000.00	30

Ahora probaremos nuestra B.D. con una consulta específica a la tabla **productos**

```
SELECT * FROM productos WHERE codigo = 'P001';
```

El resultado

productos				
codigo	nombre	categoria	precio	stock
P001	Notebook Lenovo	Tecnología	450000.00	12

Podemos probar el impacto del uso de índices

```
CREATE INDEX idx_codigo ON productos(codigo);
```

¿QUE RESULTADO VISUAL OBTUVISTE?

No produce un resultado visual directo, como lo hace una consulta SELECT. Sin embargo, lo que sucede internamente en el sistema gestor de bases de datos (SGBD) es que se crea un índice llamado **idx_codigo** sobre la columna **codigo** de la tabla **productos**.

Este índice **idx_codigo** permite que, al ejecutar consultas como **SELECT * FROM productos WHERE codigo = 'P001'**; lo que es especialmente útil cuando la tabla crece y contiene miles o millones de registros, de esta forma el motor de la base de datos puede localizar el registro rápidamente sin tener que escanear toda la tabla, lo que se conoce como **table scan**.

El índice no altera la tabla ni sus datos visibles, ni se tiene que modificar las consultas ya que el SGBD decide automáticamente usar el índice si mejora el rendimiento.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

¿QUERÉS TENER UN RESULTADO VISUAL?

Para eso podemos utilizar lo visto en clases anteriores con herramientas como **EXPLAIN** o **EXPLAIN ANALYZE** para observar si el índice **idx_codigo** fue utilizado en la búsqueda.

Vayamos al SQL Workbench o algún cliente SQL compatible para verificar si el índice **idx_codigo** se está utilizando.

EXPLAIN SELECT * FROM productos **WHERE** codigo = 'P001';

El resultado

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	productos	const	idx_codigo	idx_codigo	32	const	1	Using index

Recordando clases anteriores, leemos la tabla generada con el comando **EXPLAIN**

type = const: indica que se está usando una búsqueda muy eficiente (clave primaria o índice único).

key = idx_codigo: el índice **idx_codigo** fue seleccionado para optimizar la búsqueda.

rows = 1: el motor de la base de datos estima que devolverá un solo registro (óptimo).

Using index: no necesita leer más datos de disco; todo está en el índice.

BASE DE DATOS II

CONTENIDOS Y ACTIVIDADES

SOLUCIÓN DE LOS TRABAJOS PRÁCTICOS

ACTIVIDAD DE PRÁCTICA N° 5: CON ÍNDICE

Teniendo de base la colección de datos **bd_ejemplo_indices** creada anteriormente

Creamos el índice **idx_codigo** en la tabla **productos**, aplicando el índice sobre la columna **codigo**.

```
CREATE INDEX idx_codigo ON productos(codigo);
```

¿QUE RESULTADO VISUAL OBTUVISTE?

Aquí al igual que en el ejemplo de la actividad práctica número 4, no se modifica visualmente la tabla **productos** ni cambia su contenido.

El índice **idx_codigo** creado no es visible en los datos de la tabla, sino que es una estructura auxiliar que el sistema gestor de base de datos utiliza internamente para acelerar las búsquedas por la columna **codigo**.

Luego de la tabla **productos** con el índice **idx_codigo** creado buscamos todos registros que sean **igual** a **P001**

```
SELECT * FROM productos WHERE codigo = 'P001';
```

El resultado

codigo	nombre	categoria	precio	stock
P001	Notebook Lenovo	Tecnología	450000.00	12

IMPORTANTE

Para poder realmente darnos cuenta el valor del uso de los índices, tenemos que hacer pruebas con bases de datos que contengan muchos registros, en pequeños registros la diferencia de tiempo de respuesta pasará casi inadvertida.

¡ATENCIÓN!

Cómo ayuda en el material complementario de esta clase encontrarás dos archivos **.SQL** listos para trabajar.